

AN EFFICIENT MEMORY-BASED FFT ARCHITECTURE

Chao-Kai Chang, Chung-Ping Hung, and Sau-Gee Chen

Department of Electronics Engineering & Institute of Electronics
National Chiao-Tung University
1001 Ta Hsueh Rd, Hsinchu, Taiwan, ROC
sgchen@cc.nctu.edu.tw; asuma.ee90g@nctu.edu.tw

ABSTRACT

This paper proposes an efficient memory-based radix-2 FFT architecture, which greatly improves the memory-based FFT [5], [6] by reducing 50% memory size requirement, while maintaining a simple address generator. Specifically the memory size is reduced to 1.25N words. In addition, the multiplier utilization rate is 100%.

1. INTRODUCTION

FFT is applied to virtually every field of signal processing applications. Particularly, the OFDM technique [1] based on FFT, is recognized as the most efficient broadband wireline and wireless modulation technique [2], [3] at the time being and future to come. Because of high throughput rate demand of an OFDM system, an efficient FFT processor is required for real-time operations. FFT architectures can be categorized as two types: the pipeline architectures such as [4] and memory-based architectures, [5], [6]. Pipeline architectures bring the regularity of VLSI implement into full play and get higher performance by using more processing elements (PE), than the memory-based architectures. Memory-based architectures generally use only one butterfly PE (or more than one to establish parallelism), some memory blocks to store input or intermediate data, and a sophisticated control unit to handle memory read/write and data flow direction. In general, memory-based FFT processors can satisfy most of the applications including the mentioned OFDM communication systems, based on the state-of-art technology.

The memory-based FFT processors have the advantage of simple address generator design [5], [6] and overlapped data loading and FFT operations between two different FFT computations [6], at the cost of 1.5N [5] or 2.5N [6] memory sizes. Architecture of [6] has a better timing performance and higher multiplier utilization ratio (i.e., 100%) than [5]. This paper improves the FFT architectures of [5], [6] by reducing the memory size to only 1.25N. The reduction does not affect the performance of the original design, with respect to throughput rate (the same as that of [6]), multiplier utilization (the same as that of [6]), and simplicity of address generator.

2. MEMORY-BASED FFT PROCESSOR OF [5], [6]

In [5], [6], two types of FFT processor architecture based on the same design idea were proposed, which we call them Type I [5] and Type II [6] structures for short. Both of them were designed to implement radix-2 decimation-in-frequency (DIF) FFT processing as shown below:

$$X[2r] = \sum_{n=0}^{(N/2)-1} (x[n] + x[n + (N/2)]) W_{N/2}^{nr} \quad (1)$$

$$X[2r+1] = \sum_{n=0}^{(N/2)-1} (x[n] - x[n + (N/2)]) W_{N/2}^{nr} W_{N/2}^{nr} \quad (2)$$

$r=0,1,\dots,(N/2)-1$

In Type I FFT structure, as shown in Figure 1, N -point data is separated into two $N/2$ -point parts (i.e., the first half and the second half), and loaded into RAM-1 and RAM-2, respectively. As suggested in eq. (1) and eq. (2), in the beginning of an FFT computation, the FFT structure picks up $x[n]$ and $x[n + (N/2)]$ from RAM-1 and RAM-2, respectively, and feed them to the butterfly PE. As shown in Figure 1, the butterfly PE consists of two subblocks: one is for the addition operation, while the other is for the subtraction of the input data. As suggested by eq. (1) and eq. (2) output data from the addition subblock do not have to be multiplied by twiddle factors and can be directly written back to proper memory locations. On the other hand, output data from the subtraction subblock are stored in RAM-3 temporarily, and will be multiplied by twiddle factors later when all the addition/subtraction operations in the butterfly PE are completed. In the first stage of an N -point FFT, outputs from the addition subblock are written sequentially back to first half part of RAM-1. After completion of the butterfly operations, the intermediate data in RAM-3, produced by the subtraction operations in the butterfly PE, are fed into the multiplier subblock for twiddle factor multiplication, and then sequentially written back to the second half parts of RAM-1 and RAM-2. In the 2nd, 3rd, and the remaining FFT stages, correspondingly RAM-1 and RAM-2 are partitioned into to four subparts, eight subparts, and so on, for the writing back of the butterfly PE outputs to the RAM memories. Butterfly operations and twiddle factor multiplications are done in different phases for different FFT stages. Under this condition, the multiplier utilization ratio is 50%. This architecture has low complexity in control unit, because that the connections between memory blocks and PEs are sim-

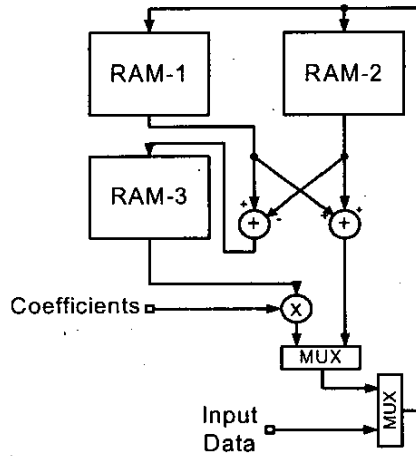


Figure 1. Architecture of Type I FFT processor [5].

ple, but at the cost of memory size (due to non-in-place computation). It needs a memory size of $1.5N$ words [5].

The Type II structure [7], as shown in Figure 2, is an improved version of Type I structure. Here, RAM-1 and RAM-2 in Type I structure are duplicated that facilitates the loading of the next N FFT data points into the other set of RAM-1 and RAM-2 memories in parallel with the current N -point FFT operations. Basically, Type II structure is the same as the Type I structure, except that two N -point FFT computations are executed simultaneously. The structure achieves 100% multiplier utilization, by interleaving the butterfly PE operations and twiddle-factor multiplications for the first and second FFT computations. However, in this case the total memory size increases to $2.5N$ words, which is much higher than the Type I structure. In order to reduce the memory size and simultaneously maintains the merits of Type-I and Type-II structures, the next section proposes a new memory-based structure that is much more efficient.

3. THE NEW IMPROVED FFT STRUCTURE

Without loss of generality, let's take a radix-2 8-point FFT for example. As shown in Figure 3, after the first FFT stage, the output data is split into two independent parts, i.e., the upper and lower parts. Therefore, we can process the first-stage FFT operations by following the configuration of Type-I structure. On the other hand, for the rest of the FFT stages, we can treat the computations as two independent $N/2$ -point FFT's, which are to be executed simultaneously. Figure 4 depicts the new improved FFT structure. In this case, for the 1st-stage FFT computation, RAM-1 and RAM-2 (each of $N/4$ words) are combined as a single memory block like RAM-1 of the Type I

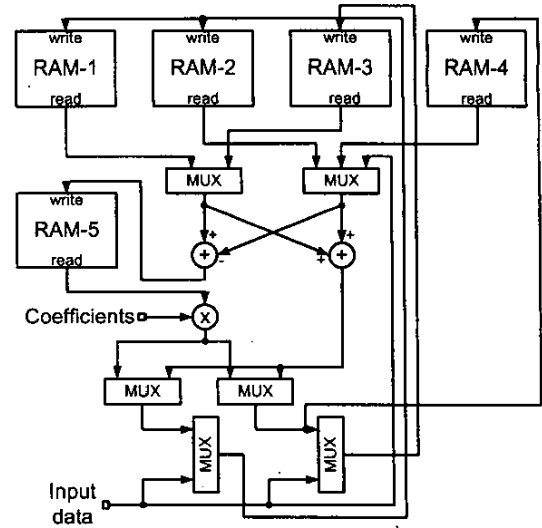


Figure 2. Architecture of Type II FFT processor [6].

structure, while RAM-3 and RAM-4 (each of $N/4$ words), are combined as a single memory block like RAM-2 of the Type I structure. For the rest of the FFT stages, similar configuration to Type II structure is adopted, except that the data path and control circuit are a little different from the Type II structure.

The detailed mechanism of the new improved FFT architecture is summarized in the following execution steps:

Step 1. Load half of input data;

Load $x[0], x[1], \dots, x[N/4-1]$ to RAM-1;
Then, load $x[N/4], x[N/4+1], \dots, x[N/2-1]$ to RAM-2.

Step 2. Process the first stage butterflies, including add, subtract, and coefficient multiply;

Read $x[k]$ from address k of RAM-1, and $x[N/2+k]$ from external input buffer and send them to butterfly PE, and perform $a[k] = x[k] + x[N/2+k]$,
 $b[k] = (x[k] - x[N/2+k]) \times W_N^k$, $k=0, 1, 2, \dots, N/4-1$.

Store $a[r]$ in address r of RAM-1, and
 $b[r]$ in address r of RAM-3,
 $r=0, 1, 2, \dots, N/4-1$.

Then, read $x[k]$ from address $k-N/4$ of RAM-2, and $x[N/2+k]$ from external input buffer and send them to butterfly PE, and perform

$a[k] = x[k] + x[N/2+k]$, $b[k] = (x[k] - x[N/2+k]) \times W_N^k$.
 $k=N/4, N/4+1, N/4+2, \dots, N/2-1$.

Store $a[r]$ in address $r-N/4$ of RAM-2, and
 $b[r]$ in address $r-N/4$ of RAM-4,
 $r=N/4, N/4+1, \dots, N/2-1$.

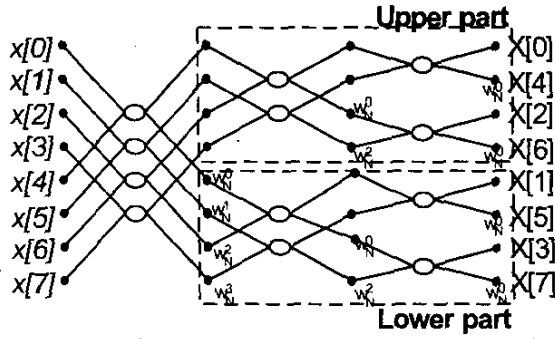


Figure 3. Signal-flow graph of an 8-point FFT. After the first FFT stage, data flow is split into two independent parts.

Step 3. Add and subtract data of upper-part butterflies;

Read $a'[k]$ from address k of RAM-1 and $b'[k]$ from address k of RAM-2 and send them to butterfly PE, and perform $a[k] = a'[k] + b'[k]$, $b[k] = a'[k] - b'[k]$, $k=0,1,2,\dots,N/4-1$.

Store $a[r]$ in address r of RAM-1, $r=0,1,2,\dots,N/8-1$.

Store $a[r+N/8]$ in address r of RAM-2, $r=0,1,2,\dots,N/8-1$.

Store $b[r]$ in address r of RAM-5, $r=0,1,2,\dots,N/4-1$.

Step 4. Complete upper-part butterflies with coefficient multiplications and simultaneously add and subtract data of lower-part butterflies;

$$c[k] = b[k] \times W_{N/2}^k, k=0,1,2,\dots,N/4-1.$$

Store $c[r]$ in address r of RAM-1, $r=0,1,2,\dots,N/8-1$.

Store $c[r+N/8]$ in address r of RAM-2, $r=0,1,2,\dots,N/8-1$.

Read $a'[k]$ from address k of RAM-3 and $b'[k]$ from address k of RAM-4 and send them to butterfly PE, and perform $a[k] = a'[k] + b'[k]$, $b[k] = a'[k] - b'[k]$, $k=0,1,2,\dots,N/4-1$.

Store $a[r]$ in address r of RAM-3, $r=0,1,2,\dots,N/8-1$.

Store $a[r+N/8]$ in address r of RAM-4, $r=0,1,2,\dots,N/8-1$.

Store $b[r]$ in address r of RAM-5, $r=0,1,2,\dots,N/4-1$.

Step 5. Complete coefficient multiplications of lower-part butterflies of one stage and simultaneously add and subtract data of upper-part butterflies of the next stage;

$$c[k] = b[k] \times W_{N/2^{t-1}}^k, k=0,1,2,\dots,N/4-1, t=3,4,\dots,\log(N).$$

Store $c[r]$ in address r of RAM-3, $r=\{y[y/(N/2^t)] \bmod(2)=0\}$, $t=3,4,\dots,\log(N)$.

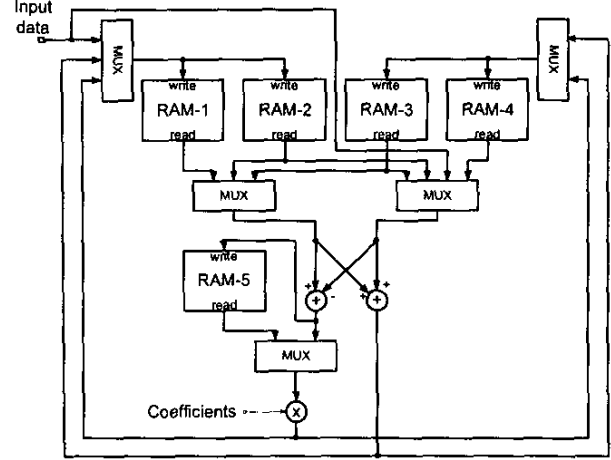


Figure 4. The new improved architecture (omitting control signal).

Store $c[r]$ in address $r-N/2^t$ of RAM-4, $r=\{y[y/(N/2^t)] \bmod(2)=1\}$, $t=3,4,\dots,\log(N)$.

Read $a'[k]$ from address k of RAM-1 and $b'[k]$ from address k of RAM-2 to butterfly PE, and perform $a[k] = a'[k] + b'[k]$, $b[k] = a'[k] - b'[k]$, $k=0,1,2,\dots,N/4-1$.

Store $a[r]$ in address r of RAM-1, $r=\{y[y/(N/2^t)] \bmod(2)=0\}$, $t=4,5,\dots,\log(N)$.

Store $a[r]$ in address $r-N/2^t$ of RAM-2, $r=\{y[y/(N/2^t)] \bmod(2)=1\}$, $t=4,5,\dots,\log(N)$.

Store $b[r]$ in address r of RAM-5, $r=0,1,2,\dots,N/4-1$.

Step 6. Complete coefficient multiplying of upper part butterflies of one stage and simultaneously add and subtract data of lower part butterflies of the same stage;

$$c[k] = b[k] \times W_{N/2^{t-1}}^k, k=0,1,2,\dots,N/4-1, t=4,5,\dots,\log(N).$$

Store $c[r]$ in address r of RAM-1, $r=\{y[y/(N/2^t)] \bmod(2)=0\}$, $t=4,5,\dots,\log(N)$.

Store $c[r]$ in address $r-N/2^t$ of RAM-2, $r=\{y[y/(N/2^t)] \bmod(2)=1\}$, $t=4,5,\dots,\log(N)$.

Read $a'[k]$ from address k of RAM-3 and $b'[k]$ from address k of RAM-4 to butterfly PE, and perform $a[k] = a'[k] + b'[k]$, $b[k] = a'[k] - b'[k]$, $k=0,1,2,\dots,N/4-1$.

Store $a[r]$ in address r of RAM-3, $r=\{y[y/(N/2^t)] \bmod(2)=0\}$, $t=4,5,\dots,\log(N)$.

Store $a[r]$ in address $r-N/2^t$ of RAM-4, $r=\{y[y/(N/2^t)] \bmod(2)=1\}$, $t=4,5,\dots,\log(N)$.

Store $b[r]$ to address r of RAM-5, $r=0,1,2,\dots,N/4-1$.

Table 1. Performance comparison of the new design and the existing designs.

	Memory size	Average throughput rate	Multiplier Utilization
Improved	1.25N	$\frac{2}{N \log(N) + N}$	100%
Type I [5]	1.5N	$\frac{1}{N \log(N) + N/2}$	50%
Type II [6]	2.5N	$\frac{2}{N \log(N) + N}$	100%

Step 7. Repeat Step 5 and Step 6 for the following stages until the last stage.

In the processing of the first-stage FFT, output data from the butterfly subtraction subblock need not to be stored in RAM-5. Instead, they are directly multiplied by the twiddle factors and written back to memory. After processing the first-stage FFT, the FFT structure proceeds to process the upper part of the intermediate output data stored in from RAM-1 and RAM-2, and simultaneously process the lower part of the intermediate output data stored in RAM-3 and RAM-4, using the similar operation configuration of Type II structure. Whenever, there is one part of intermediate data is fed into butterfly PE, the other part of intermediate data is multiplied by twiddle factor. Since RAM-5 always sends one intermediate data point to multiplier and receives one intermediate data point from butterfly PE to the same address, no data hazard is caused in RAM-5.

This architecture has 5 memory blocks, each of size $N/4$ words. As such, the total memory size is $1.25N$, which is only half that of Type II architecture.

4. PERFORMANCE EVALUATION

According to the discussed operation steps, we evaluate and compare the architectural performances of the new structure, Type I and Type II structures, based on the required total number of machine cycles, throughput rate, multiplier utilization ratio and memory size. For the evaluation, a few assumptions are made. First, the memory units support two-phase independent accesses: read phase and write phase. Secondly, maximum period between data read and write is defined as one cycle. We can figure out that critical path of this architecture contains one butterfly PE and one multiplier, that is bounded by the Step 2 mentioned in Section 3. As a result, for an N -point FFT, the required machine cycles are contributed by:

Step1. $N/2$ cycles.

Step2. $N/2$ cycles.

Step3. $N/4$ cycles.

Step4. $N/4$ cycles.

Step5. $(N/4) * (\log(N)-2)$ cycles.

Table 2. Required FFT clock rates for some OFDM communication standards using the improved FFT processor.

Communication systems	FFT size(operating frequency in MHz)
DAB	2048(12),1024(11),512(10),256(9)
DVB-T	8192(56),2048(48)
802.11a	64(70)

Step6. $(N/4) * (\log(N)-2)$ cycles.

Hence, without considering data output overhead, total latency of an N -point FFT is $N/2 + (N/2) * \log(N)$ cycles. Note that the period of a machine cycle is bounded by the critical path containing one butterfly PE, one multiplier, and several memory access overhead, and they are the same for Type I, Type II, and the proposed structures.

The Type II structure needs $N + N * \log(N)$ cycles to compute two N -point FFT's. Therefore, in average, it has the same time performance as the proposed architecture, but it requires double memory size that of the proposed structure. Performance comparison is summarized in Table 1.

Based on the proposed structure, the required clock cycles for some OFDM communication systems such as DAB, DVB-T, and 802.11a, are summarized in Table 2.

5. CONCLUSION

The proposed new structure significantly reduces the memory size, while maintains the same speed performance, compared with its predecessors [6]. The future works will be enhancing the proposed architecture for variable length FFT's that suits for different OFDM systems, and realizing the processor for particular applications.

6. REFERENCES

- [1] Richard van Nee and Ramjee Prasad, *OFDM Wireless Multimedia Communications*, Artech House, 2000
- [2] IEEE Standard 802.16-2001 IEEE Standard for Local and Metropolitan area networks Part 16: Air Interface.
- [3] IEEE Standard 802.11a-1999 Part 11: wireless LAN medium access control (MAC) and physical layer (PHY) specifications.
- [4] Shousheng He and Mats Torkelson, "A New Approach to Pipeline FFT Processor," Parallel Processing Symposium, The 10th International, pp. 766-770, 1996.
- [5] Ching-Hsien Chang, Chin-Liang Wang, and Yu-Tai Chang, "A novel memory-based FFT processor for DMT/OFDM applications," Acoustics, Speech, and Signal Processing, 1999. Proceedings. 1999 IEEE International Conference on, Volume:4, pp. 1921-1924, 1999
- [6] Chin-Liang Wang and Ching-Hsien Chang, "A DHT-based FFT-IFFT processor for VDSL transceivers," Acoustics, Speech, and Signal Processing, 2001. Proceedings. 2001 IEEE international Conference on, Volume:2, pp. 1213-1216, 2001.